COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S3
DATE PERFORMED:	9/13/2026	**DATE SUBMITTED:**	9/15/2026
NAME:	Dimaguiba, Lorenzo James M.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

HANDS-ON ACTIVITY NO. 8.1 SORTING ALGORITHMS PT. 2

1. PROCEDURE OUTPUT

MAIN DRIVER CODE

Dimaguiba_Sorting > SORT2 > main.cpp > main()

```cpp
#include <iostream>
#include "sort2.h"
#include <cstdlib>
#include <time.h>

const int maxSize = 100;

int main(){
    int dataArr[maxSize];
    srand(time(0));
    for (int i = 0; i < maxSize; i++ ){
        dataArr[i] = rand() % 10 + 1;
    }

    std::cout<<"Unsorted list:"<<std::endl;
    for (int i = 0; i < maxSize; i++ ){
       std::cout<<dataArr[i]<<" ";
    }

    std::cout<<"\n Sorted List:"<<std::endl;

    //Remove comment to test:

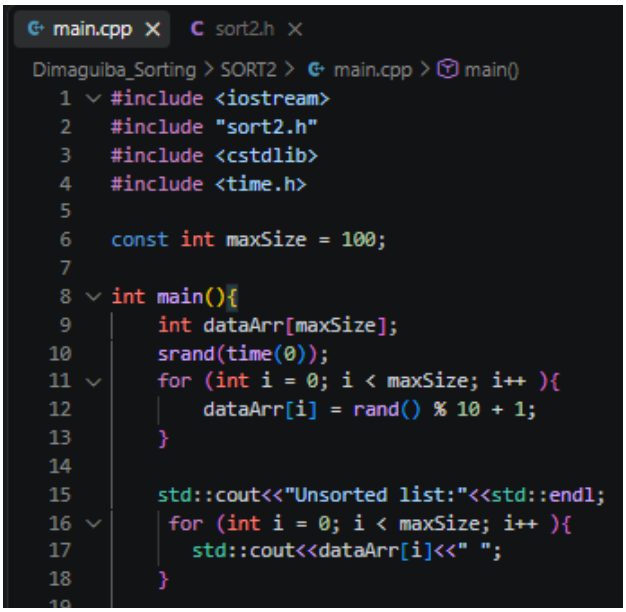
    //shellSort(dataArr, maxSize);
    //merge_sort(dataArr, 0, maxSize - 1);
    //quickSort(dataArr, 0, maxSize - 1 );

    for (int i = 0; i < maxSize; i++ ){
       std::cout<<dataArr[i]<<" ";
    }

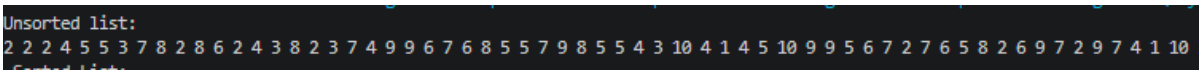
    return 0;
}

```

CODE + CONSOLE SCREENSHOT



OUTPUT:



OBSERVATIONS

Much like before, using the same libraries (cstdlib and time), we first find the seed, with “srand” that is connected to the “time” function. Generating values based on that seed (time), with “rand’ assigning to each element of the array. The other for loops are purely for displaying the array.

Table 8-1. Array of Values for Sort Algorithm Testing

CODE + CONSOLE SCREENSHOT

```
template <typename T>
void shellSort(T arr[], int arrSize){ //shellSort(array, size)
    for(int interval = arrSize/2; interval > 0; interval /= 2) { //for interval i <- size/2n down to 1
        for(int i = interval; i < arrSize; i++){ //for each interval "i" in array

            int temp = arr[i];
            int j;

            for(j = i; j >= interval && arr[j - interval] > temp; j -= interval){ //sort all the elements at interval "i"

                arr[j] = arr[j - interval];
            }

            arr[j] = temp; // insert temp

        }
    }
}
```

```
24      shellSort(dataArr, maxSize);
25      //merge_sort(dataArr, 0, maxSize - 1);
26      //quickSort(dataArr, 0, maxSize - 1 );
```

OUTPUT:

```
Unsorted list:
3 4 6 10 3 7 5 4 2 3 9 3 5 6 5 7 8 1 8 2 2 6 7 7 7 1 5 4 2 5 8 7 2 8 2 8 1
 Sorted List:
1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4
```

OBSERVATIONS

The shell sort is reminiscent of the bubble sort for some, especially with its process of moving up the array and comparing 2 values as it goes. The only difference this time is that there is a set gap between the 2 indices compared. The gap can be set to a value, but in good practice, the most common being cut in half (/2n) for each pass. So it compares, then based on the condition (greater than or less than, depending on the order) it performs a swap. After each pass, the gap gets smaller and smaller (/2n) till it reaches the loop's condition. By that point the gap would've compared those adjacent to it, finalizing the array's order to either ascending or descending order. It has a time complexity of O(n log n) at its best and O(n^2) at its worst.

Table 8-2. Shell Sort Technique

CODE + CONSOLE SCREENSHOT

```
24      //shellSort(dataArr, maxSize);
25      merge_sort(dataArr, 0, maxSize - 1);
26      //quickSort(dataArr, 0, maxSize - 1 );
```

```
42  template <typename T>
43  void merge(T arr[], int left, int middle, int right){
44
45      int n1 = middle - left + 1;
46      int n2 = right - middle;
47
48      //temporary, creates 2 different arrays
49      T L[n1];
50      T R[n2];
51
52      //copy to temporaries
53      for(int i = 0; i < n1; i++){
54          L[i] = arr[left + i];
55      }
56
57      for(int j = 0; j < n2; j++){
58          R[j] = arr[middle + 1 + j];
59      }
60
61      int i = 0;
62      int j = 0;
63      int k = left;
64
65      // Merge together
66      while(i < n1 && j < n2){
67          if(L[i] <= R[j]){
68              arr[k] = L[i];
69              i++;
70          }
71          else{
72              arr[k] = R[j];
73              j++;
74          }
75          k++;
76      }
77
78       //Copy remaining L
79      while (i < n1) {
80          arr[k] = L[i];
81          i++;
82          k++;
83      }
84
85      // Copy remaining R
86      while (j < n2) {
87          arr[k] = R[j];
88          j++;
89          k++;
90      }
91
92  }
```

```cpp
template <typename T>
void merge_sort(T arr[], int left, int right){
    //If N=1, Arr is already sorted
    if( left >= right){
        return;
    }

    int middle = (left + right)/2;

    merge_sort(arr, left, middle); //Call merge_sort(Arr,left,middle) =>sort first half recursively
    merge_sort(arr, middle + 1, right); //Call merge_sort(Arr,middle+1,right) => sort second half recursively
    merge(arr, left, middle, right); //Call merge(Arr, left, middle, right) to merge sorted arrays in above steps.

}
```

OUTPUT:

```
Unsorted list:
2 2 8 7 1 4 1 1 10 7 7 2 7 3 7 10 9 5 6 6 6 2 5 2 9 9 4 8 5
 Sorted List:
1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 3 3
```

OBSERVATIONS

May not be the fastest, but is the most stable out of the entire bunch. It follows a divide-and-conquer model. How this algorithm works sounds simpler in words but a little bit more complex in terms of code. But complexity out of the way, how it operates is very interesting. What it does once it's called is that it finds the middle of the entire array. It splits it in half by recurring itself for each half. One to the left and one to the right. Finally, the helper function (merge) temporarily creates 2 arrays pertaining to each side (L and R). It then goes through a series of comparisons. See, once they've been broken down to their last pairs, it compares one element from one side to the other, then merges them together in order, going back to its original size, then once both sides have been sorted, they are once again merged, returning as one whole array. It has a time complexity of O(n log n) at its best and its worst. In some versions, it makes use of the "vector" library, but I decided to keep it barebones without it.

Table 8-3. Merge Sort Algorithm

CODE + CONSOLE SCREENSHOT

```
24      //shellSort(dataArr, maxSize);
25      //merge_sort(dataArr, 0, maxSize - 1);
26      quickSort(dataArr, 0, maxSize - 1 );
27
```

```
123  template <typename T>
124  int partition(T arr[], int low, int high){
125
126      //declare pivot at last index
127      int pivot = arr[high];
128
129      //lagging tracker
130      int i = low - 1;
131
132      //leading traversal
133      for(int j = low; j < high; j++){
134          if(arr[j] <= pivot){
135              i++;
136              std::swap(arr[i], arr[j]);
137
138          }
139      }
140
141      //move pivot to element greater than i then return position
142      std::swap(arr[i + 1], arr[high]);
143      return i + 1;
144
145  }
146
147  template <typename T>
148  void quickSort(T arr[], int low, int high){
149      if (low < high){
150          int pivot = partition(arr, low, high); //pivot = partition (A,low,high);
151
152          quickSort(arr, low, pivot - 1); //quicksort(A,low,pivot-1)
153          quickSort(arr, pivot + 1, high); //quicksort(A,pivot+1,high)
154      }
155  }
156
157
```

OUTPUT:

```
Unsorted list:
8 8 3 4 2 1 4 5 9 2 7 6 1 6 7 5 1 9 8 7 7 9 8 3 6 5 9 6 4 3
 Sorted List:
1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 3 3
```

OBSERVATIONS

Perhaps the fastest of the group, the quicksort is best applied in large datasets with its speedy performance. It functions mainly through having 3 different tracking variables, the “pivot”, the swap variable (lagging variable) and the comparator variable (the leading variable). It also follows the divide-and-conquer model depending on the pivot. There are a series of conditions that it checks. Based on the pivot, if the comparing variable is greater than the pivot element, it finishes a step and goes up an index, keeping the pivot in place as well as the swapping variable. By the time the comparator is less than the pivot, it increases the swap variable an index, then begins a swap, it ends when the comparator reaches the pivot. It then recurs with itself once more, returning new pivots from both sides from behind the new pivot and the last index repeating the process till it is all sorted.

Table 8-4. Quick Sort Algorithm

Observation/Analysis:
Three new different sorting algorithms have been introduced. They all have the same goal, but the methods and functions they use are what differs from each other. Each sorting algorithm varies in terms of stability and speed. The shell algorithm compares and orders elements with a gap in between them that shrinks for each pass. Merge sort splits the array into smaller halves, orders them, and merges them again as it sees fit. Quick sort goes through a process called “partitioning”. Where it takes a reference point (pivot) and rearranges them depending on the conditions. The pivot may change and the partitioning process repeats. Out of all of them, the most stable would have to be the Merge Sort as it maintains order as they merge back into a sorted array. The fastest performer would have to be the Quick Sort algorithm when managing large amounts of data.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

EXAMPLE CODE

Dimaguiba_Sorting > SORT2 > C sort2 _supp.h > partition<T>(T [], int, int)

```
#ifndef SORT2_SUPP_H
#define SORT2_SUPP_H
#include <iostream>

template <typename T>
void insertSort(T arr[], int low, int n){

    for(int i = low + 1; i < n + 1; i++){
        int temp = arr[i];
        int j = i;
        while(j > low && arr[j - 1] > temp){
            arr[j] = arr[j - 1];
            j--;
        }
        arr[j] = temp;
    }
}

template <typename T>
int partition(T arr[], int low, int high){

    //declare pivot at last index
    int pivot = arr[high];

    //lagging tracker
    int i = low - 1;

    //leading traversal
    for(int j = low; j < high; j++){
        if(arr[j] <= pivot){
            i++;
            std::swap(arr[i], arr[j]);

        }
    }

    //move pivot to element greater than i then return position
    std::swap(arr[i + 1], arr[high]);
    return i + 1;

}

template <typename T>
void quickSort(T arr[], int low, int high){
    if (low < high){
        int pivot = partition(arr, low, high); //pivot = partition (A,low,high);

        quickSort(arr, low, pivot - 1); //quicksort(A,low,pivot-1)
        quickSort(arr, pivot + 1, high); //quicksort(A,pivot+1,high)
    }
}

```

```cpp
template <typename T>
void quickInsertSort(T arr[], int low, int high, int arrSize){
    while (low < high){

        if(high - low + 1 < arrSize){
            insertSort(arr, low, high);
            break;
        }
        else{
            int pivot = partition(arr, low, high);

            if (pivot - low < high - pivot){
                quickInsertSort(arr, low, pivot - 1, arrSize);
                low = pivot + 1;
            }
            else{
                quickInsertSort(arr, pivot + 1, high, arrSize);
                high = pivot - 1;

            }
        }
    }
}


#endif
```

DRIVER:

Dimaguiba_Sorting > SORT2 > main _supp.cpp > main()

```cpp
#include <iostream>
#include "sort2 _supp.h"
#include <cstdlib>
#include <time.h>

const int maxSize = 100;

int main(){
    int dataArr[maxSize];
    srand(time(0));
    for (int i = 0; i < maxSize; i++ ){
        dataArr[i] = rand() % 10 + 1;
    }

    std::cout<<"Unsorted list:"<<std::endl;
    for (int i = 0; i < maxSize; i++ ){
      std::cout<<dataArr[i]<<" ";
    }


    std::cout<<"\n Sorted List:"<<std::endl;

    quickInsertSort(dataArr, 0, maxSize -1, maxSize);

    for (int i = 0; i < maxSize; i++){
       std::cout<<dataArr[i]<<" ";
    }

    return 0;
}
```

OUTPUT:

```
t-bvjmwprr.vqv' '--stderr=Microsoft-MIEngine-Error-3b4vxwhm.kdt' '--pid=Microsoft-MIEngine-Pid-1pwg5edy.vy0' '--dbgExe=C:\msys64\ucrt64\bin\gdb.exe' '--interpreter=mi'
Unsorted list:
2 3 5 10 5 4 5 7 2 6 7 7 7 5 3 2 3 1 3 1 9 6 9 1 9 9 6 8 3 3 1 5 7 4 6 6 5 4 4 2 10 5 7 4 2 3 6 1 7 1 8 2 9 8 4 6 2 5 9 10 10 1 7 5 5 7 10 5 9 9 4 2 3 8 7 9 7 9 2 5 7 10 5 1 8 6 3 6 9 4 10 10 2 8 2 2 4 3 5 6
 Sorted List:
1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4 5 5 5 5 5 5 5 5 5 5 5 5 5 5 6 6 6 6 6 6 6 6 6 6 6 7 7 7 7 7 7 7 7 7 7 7 8 8 8 8 8 8 9 9 9 9 9 9 9 9 9 9 9 10 10 10 10 10 10 10 10
```

ANSWER TO QUESTION

Problem 1: Can we sort the left sub list and right sub list from the partition method in quick sort using other sorting algorithms? Demonstrate an example

- **Yes, this example demonstrates a quick-insert-sort hybrid. It combines elements from both sorting algorithms. It runs a modified version of the insertion sort algorithm with an extra parameter as not all arrays have to start at index 0. It starts off by determining the position of the array being sorted, next it checks the number of elements, if it is less than the array size, it stops using the quick sort algorithm to switch to the insertion sort algorithm. Otherwise it continues on its way with quicksort.**

CODE + OUTPUT

```
29      //Question 2:
30      int testArr[15] = {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74};
31
32
33      std::cout<<"\nCurrent Order:"<<std::endl;
34      for (int i = 0; i < 15; i++ ){
35        std::cout<<testArr[i]<<" ";
36      }
37
38      quickSort(testArr, 0, 15 - 1);
39
40      std::cout<<"\nNew Order:"<<std::endl;
41      for (int i = 0; i < 15; i++){
42        std::cout<<testArr[i]<<" ";
43      }
44
45      return 0;
46  }
```

OUTPUT:

```
Current Order:
4 34 29 48 53 87 12 30 44 25 93 67 43 19 74
New Order:
4 12 19 25 29 30 34 43 44 48 53 67 74 87 93
PS C:\Users\shosh\OneDrive\Documents\VSCODES>
```

ANSWER TO QUESTION

Problem 2: Suppose we have an array which consists of {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74}. What sorting algorithm will give you the fastest time performance? Why can merge sort and quick sort have O(N • log N) for their time complexity?

- **Although not the most stable but is the fastest option so far, the quicksort was used due to it having a faster completion time. Why they can have a time complexity of O(n log n) is because**

they tend to divide arrays into halves or partition them into parts and they begin to sort and process all the data before merging again with a given amount of elements being processed at every step (n).

Observation/Analysis:

Based on the recent activity, depending on the algorithm, it is possible to combine different sorting algorithms together. The Quick-Insertion-Sort hybrid function is proof of this concept that makes the algorithm a bit more efficient. It changes sorting algorithms when the sublists get to an appropriate size into the insertion sort. The best sorting algorithm to use is subjective and varies on the situation. Some data is better sorted according to its type, the volume to be sorted, and the time it takes to sort.

Concluding statement / Feedback: How well did you think you did in this activity? What are your areas for improvement?

I would be lying if I said that I understood it all. Although I know how to explain and visualize how the sorting algorithms work, applying it into code was more complex than I thought. The given algorithms in the procedure may give the overall directions of what's needed in the code, but I find it incomplete. It doesn't mention anything about the "merge" or "partition" functions and only references them. So I had to scour through different resources to figure out those functions. I'm still a bit confused on how to relate them to their time complexities as they are structured in an unfamiliar way for me.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explanation of concepts**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used a handful of resources that cover the sorting algorithms, and being able to navigate through all the jargon and ask for clarifications on some of them is a big help.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance

with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.